

Linux Fundamentals Practical

Lab Module 1: Directory Navigation & Location

1. pwd — Print Working Directory

- **Purpose:** Identifies the exact absolute directory path where your shell session is currently positioned.
- **Explanation :** Like checking the GPS on your phone to see your exact street address inside the computer's folder structure.
- **Execution Syntaxes & Variations:**

```
# Standard execution to display current directory  
pwd
```

```
# Display the physical directory (resolving symbolic links)  
pwd -P
```

```
# Display the logical directory (including symlinks)  
pwd -L
```

2. cd — Change Directory

- **Purpose:** Navigates between different folders and directories within the Linux filesystem.
- **Explanation :** Walking through an office doorway into another room.
- **Execution Syntaxes & Variations:**

```
# Move to a specific subfolder  
cd Downloads
```

```
# Jump straight back to the user's home directory  
cd ~  
# or simply:  
cd
```

```
# Move up one level to the parent directory  
cd ..
```

```
# Switch back to the previously visited directory  
cd -
```

3. ls — List Directory Contents

- **Purpose:** Displays files and subdirectories located inside the current or specified directory.
- **Explanation :** Opening a filing cabinet drawer to see everything stored inside.
- **Execution Syntaxes & Variations:**

```
# Standard simple list
ls

# Detailed long listing (permissions, owner, file size,
timestamps)
ls -l

# Show all files including hidden files (prefixed with a dot .)
ls -a

# Combined detailed long list with hidden files
ls -al

# Human-readable file sizes (e.g., KB, MB, GB)
ls -lh
```

Lab Module 2: File & Directory Creation, Copying & Deletion

4. touch — Create Empty File / Update Timestamp

- **Purpose:** Creates a new, blank file or updates the access and modification timestamps of an existing file.
- **Explanation :** Pulling a clean, empty sheet of paper from a drawer and writing a label on it.
- **Execution Syntaxes & Variations:**

```
# Create a single blank file
touch dell

# Create multiple blank files simultaneously
touch file1.txt file2.txt file3.txt

# Update timestamp without creating a new file if it does not
exist
touch -c existingfile.txt
```

5. mkdir — Make Directory

- **Purpose:** Creates one or more new directories (folders).
- **Explanation :** Setting up a brand-new storage box to organize files.

- **Execution Syntaxes & Variations:**

```
# Create a single directory
mkdir mydirectory
```

```
# Create nested directory trees in a single step (parent folders
created automatically)
mkdir -p project/2026/reports
```

```
# Create multiple folders at once
mkdir folder1 folder2 folder3
```

6. cp — Copy Files and Directories

- **Purpose:** Duplicates files or directories from a source location to a destination.
- **Explanation :** Making a photocopy of a document and placing the duplicate in another folder.

- **Execution Syntaxes & Variations:**

```
# Copy a single file into a directory
cp newfile Music/
```

```
# Copy and rename a file in one step
cp students.txt newstudents.txt
```

```
# Recursively copy an entire folder and all its contents
cp -r Music /Downloads
```

```
# Interactive copy (prompts before overwriting existing files)
cp -i newfile Music/
```

7. mv — Move / Rename Files and Directories

- **Purpose:** Relocates files and directories to new paths, or renames them in place.
- **Explanation :** Picking up a folder to place it in another drawer, or erasing the label on a folder and writing a new name.

- **Execution Syntaxes & Variations:**

```
# Move a file into another directory
mv newfile Documents/
```

```
# Rename a file in place
mv oldname.txt newname.txt
```

```
# Move an entire directory into another directory
mv sam Documents/
```

```
# Force move without prompting for confirmation
mv -f newfile Documents/
```

8. rm — Remove Files and Directories

- **Purpose:** Deletes files and directory trees permanently from the filesystem.
- **Explanation :** Putting papers or entire storage boxes into a permanent paper shredder.
- **Execution Syntaxes & Variations:**

```
# Remove a single file
rm newstate.txt
```

```
# Remove an entire folder and all nested contents recursively
rm -r mydirectory
```

```
# Force removal without any confirmation prompts
rm -rf old_directory
```

```
# Interactive prompt asking before each deletion
rm -i myfile.txt
```

9. rmdir — Remove Empty Directory

- **Purpose:** Removes directories only if they are completely empty.
- **Explanation :** Throwing away an empty cardboard box safely without risking deleting files inside.
- **Execution Syntaxes & Variations:**

```
# Remove an empty folder
rmdir mydirectory
```

```
# Remove nested empty directory hierarchies
rmdir -p parent/child/subchild
```

10. ln — Create Links / Shortcuts

- **Purpose:** Creates links (hard links or soft/symbolic shortcuts) pointing to existing target files.
- **Explanation :** Creating a desktop shortcut or shared reference pointer to a file located elsewhere on disk.
- **Execution Syntaxes & Variations:**

```
# Create a hard link (points directly to file data on disk)
ln state.txt lstate
```

```
# Create a soft / symbolic link (standard shortcut pointer)
ln -s /home/hemanth/state.txt /home/hemanth/Desktop/state_shortcut
```

```
# Force overwrite of existing destination link
ln -sf target.txt existing_link
```

Lab Module 3: File Inspection & Text Processing

11. cat — Concatenate and Display File Content

- **Purpose:** Reads text files sequentially and outputs their full contents directly to the terminal.
- **Explanation :** Opening a printed document and reading it from the very first line down to the bottom.
- **Execution Syntaxes & Variations:**
 - # Display entire file contents
`cat cdac.txt`
 - # Display file contents with numbered lines
`cat -n cdac.txt`
 - # Combine and display multiple files together
`cat part1.txt part2.txt`
 - # Concatenate files into a new unified output file
`cat file1.txt file2.txt > combined.txt`

12. tac — Display Content in Reverse

- **Purpose:** Outputs the contents of a file in reverse line order (the inverse of cat).
- **Explanation :** Reading a document upside-down, starting from the last row and finishing at the first row.
- **Execution Syntaxes & Variations:**
 - # Print file contents in reverse order
`tac cdac.txt`
 - # Pipe reversed contents to another utility
`tac logfile.txt | grep "ERROR"`

13. head — View Top of File

- **Purpose:** Outputs the beginning portion (first few lines) of a file or stream.
- **Explanation :** Reading only the introductory headline or first paragraph of an article.
- **Execution Syntaxes & Variations:**
 - # Display the default first 10 lines
`head linux.txt`
 - # Display a specific number of opening lines (e.g., first 5 or 18 lines)

```
head -n 5 read.sh
head -n 18 linux.txt

# Display the first N bytes of a file
head -c 50 linux.txt
```

14. tail — View End of File

- **Purpose:** Outputs the last lines of a file, with real-time stream monitoring capabilities.
- **Explanation :** Reading the signature at the bottom of a contract, or watching an active ticker update in real time.

- **Execution Syntaxes & Variations:**

```
# Display the default last 10 lines
tail linux.txt
```

```
# Display a specific number of ending lines
tail -n 15 linux.txt
```

```
# Continuously follow / monitor incoming log events in real time
tail -f /var/log/messages
```

15. more — Screen-by-Screen Pager

- **Purpose:** Displays long text files one page at a time with forward-scrolling navigation.
- **Explanation :** Reading an eBook page-by-page using the spacebar so text does not fly past your eyes.

- **Execution Syntaxes & Variations:**

```
# Open file in pager view
more linux.txt
```

```
# Display pager starting at a specific line number
more +20 linux.txt
```

```
# Set custom screen window size (e.g., 10 lines per screen)
more -10 linux.txt
```

16. grep — Global Regular Expression Print (Pattern Search)

- **Purpose:** Searches files or input streams for lines matching a specified text pattern.
- **Explanation :** A lightning-fast Ctrl+F search highlighting every line that contains your search word.

- **Execution Syntaxes & Variations:**

```
# Case-sensitive keyword search
grep "k" state.txt
```

```
# Case-insensitive search (matches 'k', 'K', 'Goa', 'goa')
grep -i "goa" state.txt

# Search recursively through all files in a folder
grep -r "error" /var/log/

# Invert search (print only lines that DO NOT contain the pattern)
grep -v "success" test.log
```

17. sort — Sort Lines of Text

- **Purpose:** Reorders lines in text files alphabetically, numerically, or in reverse.
- **Explanation :** Sorting a stack of index cards from A to Z or organizing numbers from lowest to highest.
- **Execution Syntaxes & Variations:**

```
# Standard alphabetical ascending sort
sort cdac.txt

# Reverse sort (Z to A)
sort -r cdac.txt

# Numerical sort
sort -n scores.txt

# Sort and eliminate exact duplicate lines
sort -u cdac.txt
```

18. wc — Word, Line, and Byte Count

- **Purpose:** Calculates the total number of lines, words, and byte counts in a file.
- **Explanation :** The word counter in your document editor showing total pages, lines, and words.
- **Execution Syntaxes & Variations:**

```
# Display full line count, word count, and byte count
wc state.txt

# Count lines only
wc -l state.txt

# Count words only
wc -w state.txt

# Count characters / bytes only
wc -c state.txt
```

19. diff — File Comparison Utility

- **Purpose:** Compares two files line by line and highlights all differences between them.
- **Explanation :** Placing two documents side by side under a light to highlight edited text.
- **Execution Syntaxes & Variations:**

```
# Standard line-by-line comparison
diff students newstudents

# Side-by-side comparison view
diff -y file1.txt file2.txt

# Unified format comparison (standard patch format)
diff -u config_v1.conf config_v2.conf

# Case-insensitive comparison
diff -i file1.txt file2.txt
```

20. file — Determine File Type

- **Purpose:** Inspects file headers to identify the true file format regardless of extension.
- **Explanation :** Opening a package to check what is inside instead of trusting the shipping label.
- **Execution Syntaxes & Variations:**

```
# Check file type of a text file
file state.txt

# Check file type of a folder
file Downloads

# Brief output mode (displays only the MIME/type description)
file -b image.png
```

Lab Module 4: Help Systems & Command Discovery

21. man — System Reference Manual

- **Purpose:** Opens the official full-length documentation manual for commands and configuration files.
- **Explanation :** Opening the complete manufacturer user manual for a tool.
- **Execution Syntaxes & Variations:**

```
# Open manual for the ls command
man ls

# Open manual for a specific section (e.g., section 1 for user
```



```
commands)
man 1 chmod

# Search for a keyword across all manual headers
man -k "partition"
```

22. whatis — One-Line Manual Summary

- **Purpose:** Displays a single-sentence overview of a command's function from the manual database.
- **Explanation :** Asking for an elevator pitch summary of what a tool does.
- **Execution Syntaxes & Variations:**

```
# One-line summary for ls
whatis ls

# Query multiple commands at once
whatis cp mv rm mkdir
```

23. whereis — Locate Executable, Source, and Manual Files

- **Purpose:** Locates the binary executable paths, source code, and man pages for a command.
- **Explanation :** Asking the system where an application's physical installation folders live.
- **Execution Syntaxes & Variations:**

```
# Locate all occurrences of a command
whereis ls

# Locate only the binary executable path
whereis -b ls

# Locate only the manual documentation paths
whereis -m ls
```

24. find — Search Directory Trees

- **Purpose:** Recursively searches storage directories for files matching names, sizes, or types.
- **Explanation :** The filesystem search engine hunting through folders to find missing files.
- **Execution Syntaxes & Variations:**

```
# Find files by exact name inside a folder
find Downloads

# Search for all bash scripts ending in .sh
find /home/ldapuser/ -name "*.sh"
```

```
# Search specifically for directories only
find /var/log -type d

# Find files modified within the last 24 hours
find . -mtime -1
```

25. locate — Fast Indexed Database File Search

- **Purpose:** Instantly locates files across the system using a pre-indexed filesystem database.
- **Explanation :** Looking up a file in an index registry rather than manually walking the hard drive.
- **Execution Syntaxes & Variations:**

```
# Search for file paths matching a name
locate abc.txt

# Case-insensitive locate
locate -i "report.pdf"

# Limit search output count
locate -n 5 "*.conf"
```

Lab Module 5: Archiving & Compression

26. tar — Tape Archive Utility

- **Purpose:** Bundles multiple files and folders into a single consolidated archive package.
- **Explanation :** Packing 20 separate items into a single moving box for easy transport.
- **Execution Syntaxes & Variations:**

```
# Create an archive file (-c: create, -v: verbose, -f: filename)
tar -cvf backup.tar cdac.txt state.txt

# Extract an archive package (-x: extract)
tar -xvf backup.tar

# Create a compressed gzip archive directly in one step (-z: gzip)
tar -czvf project.tar.gz /home/hemanth/Documents

# List the contents of a tar archive without extracting it (-t: list)
tar -tvf backup.tar
```

27. gzip — Compress Files

- **Purpose:** Compresses files into .gz format using Lempel-Ziv coding (LZ77).
- **Explanation :** Vacuum-sealing a package to squeeze out air and shrink storage footprint.
- **Execution Syntaxes & Variations:**

```
# Compress a file (replaces original with cdac.txt.gz)
gzip cdac.txt

# Compress with maximum compression ratio (levels 1-9)
gzip -9 largefile.log

# Keep original file while compressing
gzip -k access.log
```

28. gunzip — Decompress Gzip Files

- **Purpose:** Restores files compressed with gzip back to their original state.
- **Explanation :** Cutting open the vacuum seal to unpack the original document.
- **Execution Syntaxes & Variations:**

```
# Decompress a .gz file
gunzip cdac.txt.gz

# Force decompression even if destination exists
gunzip -f archive.txt.gz
```

29. zcat — View Compressed File Contents

- **Purpose:** Decompresses and displays text from a .gz archive directly to the screen.
- **Explanation :** Peeking inside a vacuum-sealed bag without having to cut it open.
- **Execution Syntaxes & Variations:**

```
# Read compressed text file directly on console
zcat cdac.txt.gz

# Pipe decompressed output into a search filter
zcat logfile.gz | grep "CRITICAL"
```

Lab Module 6: System Information, Utilities & Monitoring

30. ps — Report Process Status

- **Purpose:** Displays a snapshot of active computer programs and background processes.
- **Explanation :** Opening the Windows Task Manager to see which applications are running.
- **Execution Syntaxes & Variations:**

```
# Show processes running in current terminal
ps

# Standard full system process list (-e: all, -f: full format)
ps -ef

# BSD syntax showing all running user processes with CPU/Memory
usage
ps aux
```

31. jobs — Active Background Job Monitor

- **Purpose:** Lists jobs and tasks running in the background of the current shell session.
- **Explanation :** Checking your browser download queue to see which background transfers are active.
- **Execution Syntaxes & Variations:**

```
# Display background jobs
jobs

# Display background jobs with their Process IDs (PID)
jobs -l

# Display only running jobs
jobs -r
```

32. whoami — Print Current Username

- **Purpose:** Displays the effective user account currently operating the terminal.
- **Explanation :** Checking the current profile name to confirm whether you are a regular user or root.
- **Execution Syntaxes & Variations:**

```
# Print username
whoami
```

33. who — Show Logged-In Users

- **Purpose:** Lists all users currently logged into the system along with terminal names and login times.
- **Explanation :** Checking the front-desk sign-in log to see who is in the building.
- **Execution Syntaxes & Variations:**

```
# List all active users
who

# Show user count and current runlevel
who -q
```

```
# Include column headers
who -H
```

34. w — Show Who is Logged In and What They Are Doing

- **Purpose:** Provides a detailed overview of logged-in users, idle times, CPU usage, and current commands.
- **Explanation :** An office supervisor board showing who is present and what project they are typing on.
- **Execution Syntaxes & Variations:**

```
# Display full status dashboard
```

```
w
```

```
# Short format output (skips system uptime summary)
```

```
w -s
```

```
# Check status for a specific user
```

```
w hemanth
```

35. date — Display or Set System Date and Time

- **Purpose:** Prints or configures the system clock and calendar timestamp.
- **Explanation :** Checking the wall clock or resetting the digital time display.
- **Execution Syntaxes & Variations:**

```
# Display current system date and time
```

```
date
```

```
# Set system date and time manually (requires root permissions)
```

```
date -s "Tuesday 07 February 2023 12:12:30 PM IST"
```

```
# Display date formatted as YYYY-MM-DD
```

```
date "+%Y-%m-%d"
```

36. cal — Display Calendar

- **Purpose:** Renders a clean monthly or yearly calendar layout in the terminal.
- **Explanation :** Looking at a desk calendar to see what day of the week a date falls on.
- **Execution Syntaxes & Variations:**

```
# Display current month calendar
```

```
cal
```

```
# Display entire year calendar
```

```
cal 2026
```

```
# Display a specific month and year (e.g., February 2023)
cal 2 2023
```

37. bc — Basic Calculator

- **Purpose:** An arbitrary-precision numeric calculator language for mathematical computations.
- **Explanation :** A desktop desk calculator for quick additions, multiplications, and divisions.
- **Execution Syntaxes & Variations:**

```
# Launch interactive calculator shell (type 'quit' to exit)
bc
```

```
# Non-interactive calculation via pipe
echo "1254 * 5" | bc
```

```
# Floating-point math with specific decimal scale
echo "scale=2; 10 / 3" | bc
```

38. echo — Print Text to Terminal

- **Purpose:** Outputs lines of text or variable values directly to the display or redirection stream.
- **Explanation :** A digital megaphone that repeats whatever text or variable message you tell it to state.
- **Execution Syntaxes & Variations:**

```
# Print simple text
echo "Hello, World!"
```

```
# Enable backslash escape interpretations (e.g., newlines \n, tabs \t)
echo -e "Line 1\nLine 2"
```

```
# Print text without a trailing newline
echo -n "Username: "
```

39. lpr — Offload Print Job

- **Purpose:** Submits files to a connected CUPS printer spooler for physical printing.
- **Explanation :** Hitting Ctrl+P to send a digital document to the office network printer.
- **Execution Syntaxes & Variations:**

```
# Print a file to default printer
lpr report.txt
```

```
# Print to a specific destination printer name
lpr -P Office_LaserJet document.pdf
```

```
# Print multiple copies of a document
lpr -#3 final_doc.txt
```

Lab Module 7: Network & Remote Access Commands

40. ping — Test Network Connectivity

- **Purpose:** Sends ICMP ECHO_REQUEST packets to network hosts to diagnose network connectivity and latency.
- **Explanation :** Calling out "*Can you hear me?*" across the network and timing how many milliseconds it takes for an echo to return.
- **Execution Syntaxes & Variations:**
 - # Ping an IP address continuously
ping 10.182.0.1
 - # Ping with a limited packet count (e.g., send 4 packets and stop)
ping -c 4 10.182.0.1
 - # Fast interval ping (useful for testing packet loss under load)
ping -i 0.5 10.182.0.1

41. ssh — Secure Shell Remote Login

- **Purpose:** Establishes encrypted, secure remote command-line login sessions across a network.
- **Explanation :** Securely teleporting your terminal keyboard and screen onto a server in another office or country.
- **Execution Syntaxes & Variations:**
 - # Standard remote login as root user
ssh root@10.182.0.11
 - # Log in on a non-standard port (e.g., port 2222)
ssh -p 2222 admin@192.168.1.100
 - # Execute a single remote command without opening a persistent shell
ssh root@10.182.0.11 "uptime"

Hands-On Practice Assignment

Practical Lab Tasks

Task 1: Filesystem Structure & File Management

1. Identify your current working directory using `pwd`.
2. Create a new directory structure named `assignment_lab/stage1` using a single command (`mkdir -p`).
3. Navigate into `assignment_lab/stage1` and create three blank text files: `data1.txt`, `data2.txt`, and `info.txt`.
4. Write the names of 5 states into `data1.txt` using `cat > data1.txt` or `echo`.
5. Create a duplicate backup of `data1.txt` named `data1_backup.txt`.
6. Create a soft link on your home directory named `state_link` pointing to `data1.txt`.

Task 2: Data Extraction & Sorting

1. Display the contents of `data1.txt` from top to bottom using `cat`, and then in reverse order using `tac`.
2. Sort `data1.txt` in reverse alphabetical order using `sort -r`.
3. Search for state names ending with or containing the letter `a` using `grep -i`.
4. Count the total words and lines present in `data1.txt` using `wc`.

Task 3: Archival, Compression & Verification

1. Bundle the entire `assignment_lab` folder into an archive file named `assignment.tar` using `tar -cvf`.
2. Compress `assignment.tar` using `gzip` to generate `assignment.tar.gz`.
3. Use `file` to verify the compression type of `assignment.tar.gz`.
4. Preview the decompressed file structure using `zcat` or `tar -tvf`.

Task 4: System Diagnosis & Remote Monitoring

1. Check your active username with `whoami` and system uptime with `w`.
2. Calculate 1542×8 directly from the command line using `bc`.
3. Verify network reachability to `127.0.0.1` (localhost) sending exactly 3 packets with `ping`.